Character Sets (ASCII/Unicode)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Character sets and ASCII
2. Unicode
3. Calculations and comparisons
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Character Sets (J277/01 Section 1.2.4)**. This topic covers how text is represented in binary using character sets, the difference between ASCII and Unicode, and how to calculate the number of bits needed to represent a given number of characters.

Exam questions typically ask you to **state** what a character set is, **explain** the difference between ASCII and Unicode, and **calculate** the minimum number of bits needed for a character set. Questions range from 1-mark definitions to 3-mark calculations and comparisons.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and give an example where appropriate.

- **Define** what a character set is and explain its purpose.
- **Describe** how ASCII represents characters using 7 bits (128 characters) or 8 bits for Extended ASCII (256 characters).
- **Describe** how Unicode represents characters using 16 or more bits, allowing for millions of characters.
- **Explain** why Unicode was developed and its advantages over ASCII.

- **Calculate** the minimum number of bits needed to represent a given number of characters in a character set.
- **Compare** ASCII and Unicode in terms of file size, character coverage, and compatibility.

> **Specification note.** OCR J277 Section 1.2.4 explicitly requires understanding of character sets including ASCII and Unicode, and the relationship between the number of bits used and the number of characters that can be represented. The 2024 exam included a question asking for the minimum bits needed to represent 150 different characters.

Toolkit

Character set — definition

A **character set** is a collection of characters that a computer can recognise and use. Each character is assigned a unique binary code. When you press a key on a keyboard, the character set determines which binary value is stored and displayed.

Key formula — bits and characters

The number of different characters that can be represented depends on the number of bits used:

Number of characters = 2^n (where n = number of bits)

Bits (n)	Characters (2^n)	Example use
4	16	Hexadecimal digits (0-9, A-F)
7	128	Standard ASCII
8	256	Extended ASCII
16	65,536	Basic Unicode (UTF-16)
21	2,097,152	Full Unicode range

Calculating minimum bits needed

To find the **minimum number of bits** needed for a character set:

1. Find the smallest power of 2 that is greater than or equal to the number of characters.
2. The exponent is the minimum number of bits needed.

Example: For 150 characters: 2^7 = 128 (too small), 2^8 = 256 (big enough) → **8 bits needed**

ASCII vs Unicode — quick comparison

Feature	ASCII	Unicode
Bits per character	7 bits (or 8 for Extended)	16+ bits (varies by encoding)
Number of characters	128 (or 256 Extended)	Over 140,000 (and growing)
Languages supported	English only	All world languages
Special characters	Limited symbols	Emojis, mathematical symbols, historic scripts
File size	Smaller	Larger (more bits per character)

> **Exam tip:** When asked about character sets, always mention that each character has a **unique binary code**. This is the core concept that examiners look for.

1. Character sets and ASCII

Computers store everything as binary (0s and 1s). To store text, we need a way to convert each character into a binary number. A **character set** provides this mapping — it is essentially a lookup table that tells the computer which binary pattern represents which character.

1.1 What is a character set?

A **character set** is a defined list of characters that a computer recognises, where each character is assigned a unique binary code. Characters include:

- Letters (A-Z, a-z)
- Digits (0-9)
- Punctuation marks (. , ! ? etc.)
- Special symbols (@ # $ % etc.)
- Control characters (space, tab, enter, etc.)

When you type the letter 'A', the computer doesn't store the shape of the letter — it stores the binary code that represents 'A' according to the character set being used. When the text is displayed, the computer looks up the binary code and renders the correct character shape on screen.

The character set acts as a shared agreement between the sender and receiver of data. If both systems use the same character set, the text will display correctly. If they use different character sets, characters may appear incorrectly or as unrecognised symbols.

1.2 How character encoding works

The process of converting text to binary is called **encoding**, and the reverse process is called **decoding**:

- **Encoding:** When you save a document, each character is converted to its binary code and stored

- **Decoding:** When you open a document, each binary code is converted back to the character it represents
  For example, if you type "Hi", the computer encodes this as two binary values: 'H' (72 = 1001000) followed by 'i' (105 = 1101001). These binary values are stored in sequence in the file.

1.3 ASCII

ASCII (American Standard Code for Information Interchange) is the most fundamental character set. It was developed in the 1960s in the United States and uses **7 bits** to represent each character.

With 7 bits, ASCII can represent 2^7 = **128 different characters**. These 128 positions are organised into groups:

- Control characters (codes 0-31): non-printable characters like tab, newline, backspace
- Punctuation and symbols (codes 32-64): space, !, ", #, $, digits 0-9, etc.
- Uppercase letters A-Z (codes 65-90)
- More symbols (codes 91-96): [, \,], ^, _, `
- Lowercase letters a-z (codes 97-122)
- Final symbols (codes 123-127): {, |, }, ~, delete
  For example, the letter 'A' has ASCII code 65, which is **1000001** in binary. The letter 'a' has code 97, which is **1100001** in binary.

1.4 ASCII code examples

Character	Decimal	Binary (7-bit)
A	65	1000001
B	66	1000010
Z	90	1011010
a	97	1100001
0	48	0110000
Space	32	0100000

1.5 The relationship between upper and lowercase

In ASCII, there is a deliberate mathematical relationship between uppercase and lowercase letters. The difference between any uppercase letter and its lowercase equivalent is always **32**:

- 'A' = 65, 'a' = 97 (difference of 32)
- 'B' = 66, 'b' = 98 (difference of 32)
- 'Z' = 90, 'z' = 122 (difference of 32)
  This is useful for programmers because converting between cases only requires adding or subtracting 32 from the ASCII code. In binary, this difference of 32 means only one bit changes (bit 5).

1.6 Extended ASCII

Extended ASCII uses **8 bits** instead of 7, allowing 2^8 = **256 characters**. The first 128 codes (0-127) are identical to standard ASCII, ensuring compatibility. The extra 128 codes (128-255) are used for:

- Accented letters (é, ñ, ü, ç, ø)
- Additional symbols (©, ®, °, ±, ÷)
- Currency symbols (£, ¥, €)
- Box-drawing characters for text-based interfaces
- Some Greek letters and mathematical symbols

However, different organisations created different Extended ASCII versions. For example, Western European computers used one version, while Eastern European computers used another. This caused compatibility problems when sharing documents internationally, which is one reason why Unicode was developed.

> **Exam tip:** Remember that uppercase and lowercase letters have **different ASCII codes**. 'A' is 65, but 'a' is 97. The difference of 32 is consistent for all letters.

Worked Example 1 – Defining a character set

Question: State what is meant by a character set. [2 marks]

Model answer:
A character set is a collection/list of characters that a computer can recognise **(1)**. Each character is assigned a unique binary code **(1)**.

> **Exam tip:** Always mention "unique binary code" — this is the key point that distinguishes a character set from just a list of characters.

Worked Example 2 – ASCII basics

Question: State how many characters can be represented using standard 7-bit ASCII. [1 mark]

Model answer:
128 characters **(1)**.
Working: $2^7 = 128$

> **Exam tip:** Learn the key numbers: 7-bit ASCII = 128 characters, 8-bit Extended ASCII = 256 characters.

Worked Example 3 – ASCII code lookup

Question: The ASCII code for 'A' is 65. What is the ASCII code for 'C'? [1 mark]

Model answer:
67 **(1)**.
Working: A=65, B=66, C=67 (letters are sequential)

Exam tip: ASCII codes for letters are sequential. If you know one letter's code, you can work out the others by counting.

Worked Example 4 — Binary representation

Question: The ASCII code for the letter 'B' is 66. Express this in 7-bit binary. [1 mark]

Model answer:
1000010 **(1)**.
Working: 64 + 2 = 66, so binary is 1000010

Exam tip: Use the place value method: 64+2 = 66. In binary: 1000000 + 0000010 = 1000010.

2. Unicode

ASCII was designed for English text in the 1960s and cannot represent characters from other languages. As computing became global and the internet connected people worldwide, a single universal character set became essential. **Unicode** was developed to solve this problem by providing a character set that includes every character from every writing system in the world.

2.1 The problem with ASCII

ASCII has several significant limitations that became more apparent as computing went global:

- Only 128 (or 256 with Extended) characters available — not enough for most languages
- Cannot represent Chinese characters (over 50,000 in common use)
- Cannot represent Japanese (three writing systems: hiragana, katakana, kanji)
- Cannot represent Arabic, Hebrew, Hindi, Thai, Korean, or many other scripts
- No emojis, mathematical symbols, or musical notation
- Different countries created incompatible Extended ASCII versions

The incompatibility problem was particularly serious. A document created on a French computer might display incorrectly on a Greek computer because they used different Extended ASCII codes for characters 128-255. This made international communication and data exchange unreliable.

2.2 The Unicode solution

Unicode (officially the Unicode Standard) was first published in 1991. It aims to assign a unique code point to every character ever used in human writing, plus symbols, emojis, and technical characters.

Unicode assigns a unique **code point** to every character. Code points are written as U+ followed by a hexadecimal number. For example:

- U+0041 = 'A' (Latin capital letter A)
- U+03B1 = 'α' (Greek lowercase alpha)
- U+4E2D = '中' (Chinese character meaning "middle")
- U+1F600 = '😀' (grinning face emoji)

2.3 Unicode encodings

Unicode defines the code points, but there are different ways to encode these into binary. The most common encodings are:

- **UTF-8:** Variable-width encoding (1-4 bytes). ASCII characters use 1 byte, making it efficient for English text and backwards compatible with ASCII.
- **UTF-16:** Uses 2 bytes (16 bits) for most characters. Common in Windows and Java.
- **UTF-32:** Uses 4 bytes (32 bits) for all characters. Simple but uses more storage.

For GCSE, you typically only need to know that Unicode uses 16 or more bits per character, allowing far more characters than ASCII.

2.4 Unicode capacity and coverage

Using **16 bits** (2 bytes), Unicode can represent 2^{16} = **65,536 characters**. This is called the Basic Multilingual Plane (BMP) and covers most modern languages.

For characters beyond the BMP (like historical scripts and many emojis), Unicode can use up to **21 bits**, allowing over 2 million possible code points. Currently, Unicode defines over **140,000 characters** including:

- Characters from all modern languages (Chinese, Arabic, Hindi, etc.)
- Historical scripts (Egyptian hieroglyphics, cuneiform, runes)
- Mathematical and technical symbols
- Over 3,600 emojis and pictographs
- Musical notation symbols
- Braille patterns
- Currency symbols from around the world

2.5 Unicode and ASCII compatibility

Unicode was deliberately designed to be **backwards compatible** with ASCII. The first 128 Unicode characters (U+0000 to U+007F) are identical to the 128 ASCII characters:

- 'A' is code 65 in ASCII and U+0041 (= 65) in Unicode
- 'a' is code 97 in ASCII and U+0061 (= 97) in Unicode
- All standard ASCII characters have the same code values

This backwards compatibility means that ASCII text files can be read as Unicode (specifically UTF-8) without any conversion. Programs that understand Unicode can read old ASCII files perfectly.

2.6 Trade-offs: ASCII vs Unicode

Advantages of Unicode over ASCII:

- Represents all world languages in one universal standard
- Includes emojis, symbols, and historical scripts
- No compatibility issues between different countries or systems
- Future-proof — new characters can be added as needed
- Essential for international websites and applications

Disadvantages of Unicode compared to ASCII:

- Uses more bits per character (typically 16 vs 8)

- Files are larger — potentially double the size for the same text
- More processing power and memory needed
- More complex to implement in software

File size comparison: A 1000-character English document uses approximately 1KB in ASCII (1 byte per character) but 2KB in UTF-16 Unicode (2 bytes per character). For very large documents or databases, this difference in storage requirements can be significant.

Worked Example 5 – Why Unicode was developed

Question: Explain why Unicode was developed to replace ASCII. [2 marks]

Model answer:
ASCII can only represent 128/256 characters, which is not enough for all world languages **(1)**. Unicode can represent characters from all languages, plus emojis and symbols, using a larger character set **(1)**.

Exam tip: Focus on the limitation of ASCII (not enough characters) and how Unicode solves it (supports all languages).

Worked Example 6 – Unicode character count

Question: Unicode uses 16 bits to represent each character. Calculate how many different characters can be represented. [1 mark]

Model answer:
65,536 characters **(1)**.
Working: $2^{16} = 65{,}536$

Exam tip: Use the formula: characters = 2^{bits}. For 16 bits: $2^{16} = 65{,}536$.

Worked Example 7 – Comparing ASCII and Unicode

Question: State one advantage and one disadvantage of using Unicode instead of ASCII. [2 marks]

Model answer:
Advantage: Unicode can represent characters from all world languages / more characters available / includes emojis **(1)**.

Disadvantage: Unicode files are larger because more bits are used per character **(1)**.

Exam tip: The main trade-off is: Unicode = more characters but larger files. ASCII = fewer characters but smaller files.

Worked Example 8 — Backwards compatibility

Question: Explain why an ASCII text file can be opened by a program that uses Unicode. [2 marks]

Model answer:
Unicode is backwards compatible with ASCII **(1)**. The first 128 Unicode characters have the same codes as ASCII, so ASCII characters are valid Unicode characters **(1)**.

Exam tip: "Backwards compatible" means new systems can still read old data. Unicode was deliberately designed to match ASCII for the first 128 codes.

3. Calculations and comparisons

Exam questions often require you to calculate the number of bits needed for a character set, work out file sizes, or compare ASCII and Unicode in specific scenarios. This section covers the mathematical techniques you need.

3.1 The bits-to-characters relationship

The fundamental relationship between bits and characters is:

Number of characters = 2^n (where n = number of bits)

This works because each bit can be either 0 or 1, so with n bits you can create 2^n different binary patterns. Each pattern represents one character.

- 1 bit → 2^1 = 2 patterns (0, 1)
- 2 bits → 2^2 = 4 patterns (00, 01, 10, 11)
- 3 bits → 2^3 = 8 patterns (000, 001, 010, 011, 100, 101, 110, 111)

This pattern continues: each additional bit doubles the number of possible characters.

3.2 Calculating minimum bits needed

To find the minimum number of bits needed to represent a given number of characters:

1. Identify how many characters need to be represented
2. Find the smallest power of 2 that is greater than or equal to this number
3. The exponent is the number of bits needed

Key powers of 2 to memorise:

Bits	2^n	Bits	2^n
1	2	6	64
2	4	7	128
3	8	8	256
4	16	9	512
5	32	10	1024

3.3 Step-by-step calculation examples

Q: How many bits for 100 characters?

Step 1: Find powers of 2 around 100

2^6 = 64 (too small — cannot represent 100 characters)

2^7 = 128 (big enough — can represent 100 characters with 28 spare)

Answer: **7 bits**

Q: How many bits for 150 characters?

2^7 = 128 (too small)

2^8 = 256 (big enough)

Answer: **8 bits**

Q: How many bits for 500 characters?

2^8 = 256 (too small)

2^9 = 512 (big enough)

Answer: **9 bits**

Q: How many bits for exactly 256 characters?

2^8 = 256 (exactly right)

Answer: **8 bits**

3.4 File size calculations

The size of a text file depends on two factors:

- The number of characters in the file
- The number of bits (or bytes) used per character

File size (bits) = number of characters × bits per character

To convert between units, remember:

- 8 bits = 1 byte
- 1024 bytes = 1 kilobyte (KB)
- 1024 KB = 1 megabyte (MB)

Example calculation: A 2000-character document:

- ASCII (8 bits/char): 2000 × 8 = 16,000 bits = 2000 bytes ≈ 2KB
- Unicode UTF-16 (16 bits/char): 2000 × 16 = 32,000 bits = 4000 bytes ≈ 4KB

The Unicode file is **twice the size** of the ASCII file for the same text content.

3.5 Practical implications of file size

The difference in file size matters in several real-world situations:

- **Storage:** A database with millions of text records uses significantly more disk space with Unicode
- **Bandwidth:** Larger files take longer to transmit over networks
- **Memory:** Programs need more RAM to process larger text files
- **Cost:** Cloud storage and bandwidth are often charged by usage

3.6 Choosing the right character set

Use case	Best choice	Reason
English-only document	ASCII	Smaller file size, all characters supported
Multi-language website	Unicode	Must support all languages
Programming source code	ASCII (usually)	Code uses English keywords; comments may need Unicode
Social media with emojis	Unicode	Emojis require Unicode
International database	Unicode	Names and addresses from any country
Embedded system with limited memory	ASCII	Minimises memory usage

Exam tip: In calculation questions, always show your working. Even if you get the final answer wrong, you can gain marks for demonstrating the correct method. Write out both the "too small" and "big enough" calculations.

Worked Example 9 – Minimum bits calculation

Question: A character set needs to represent 150 different characters. Calculate the minimum number of bits needed. [2 marks]

Model answer:
2^7 = 128, which is less than 150 **(1)**.
2^8 = 256, which is greater than 150, so **8 bits** are needed **(1)**.

Exam tip: This exact question appeared in the 2024 OCR paper. Show both the "too small" and "big enough" calculations to demonstrate your method.

Worked Example 10 – File size comparison

Question: A text file contains 2000 characters. Calculate the file size in bytes when stored using: (a) 8-bit ASCII, (b) 16-bit Unicode. [2 marks]

Model answer:
(a) ASCII: 2000 × 8 = 16,000 bits = 16,000 ÷ 8 = **2000 bytes (1)**.
(b) Unicode: 2000 × 16 = 32,000 bits = 32,000 ÷ 8 = **4000 bytes (1)**.

Exam tip: Remember: 8 bits = 1 byte. Divide your bits answer by 8 to get bytes. To get kilobytes, divide bytes by 1024 (or 1000 for a quick estimate).

Worked Example 11 – Scenario application

Question: A company in Japan is creating a website that will display text in Japanese, English, and include emojis. Explain why Unicode would be more suitable than ASCII for this website. [3 marks]

Model answer:
ASCII can only represent English characters (128 or 256 characters) **(1)**. Japanese uses thousands of characters (kanji, hiragana, katakana) which ASCII cannot represent **(1)**. Unicode includes all Japanese characters and emojis, making it suitable for this multilingual website **(1)**.

Exam tip: Apply your answer to the specific scenario. Don't just say "Unicode has more characters" — explain why those extra characters are needed for this situation.

Worked Example 12 – Extended comparison (4-mark style)

Question: A software company is deciding whether to use ASCII or Unicode for storing customer data. The data includes names from many countries. Compare ASCII and Unicode and recommend which character set the company should use. [4 marks]

Model answer:
ASCII uses 7 or 8 bits per character and can represent 128 or 256 characters. This is only sufficient for English and some European languages. Names from countries like China, Japan, or Arabic-speaking nations cannot be stored correctly in ASCII.
Unicode uses 16+ bits per character and can represent over 140,000 characters from all world languages. While this creates larger files, it ensures all customer names can be stored accurately.
Recommendation: The company should use Unicode because the data includes names from many countries, which requires characters from multiple alphabets and scripts that ASCII cannot represent.

> **Exam tip:** For comparison questions, describe both options, then give a clear recommendation with justification. Don't sit on the fence — pick one and explain why.

Worked examples summary

The 12 worked examples above cover the main question types you will encounter on character sets. Here is a summary of the key question patterns:

Example	Question type	Key skill
1	Define (2 marks)	Character set = list + unique binary codes
2	State characters (1 mark)	7-bit ASCII = 128 characters
3	ASCII lookup (1 mark)	Letters are sequential in ASCII
4	Binary conversion (1 mark)	Convert decimal to 7-bit binary
5	Explain Unicode (2 marks)	ASCII limitation → Unicode solution
6	Calculate characters (1 mark)	2^{16} = 65,536
7	Advantage/disadvantage (2 marks)	More characters vs larger files
8	Compatibility (2 marks)	Unicode includes ASCII as first 128
9	Minimum bits (2 marks)	Find smallest $2^n \geq$ characters
10	File size (2 marks)	Characters × bits ÷ 8 = bytes
11	Scenario (3 marks)	Apply Unicode benefits to context
12	Compare + recommend (4 marks)	Both sides + clear recommendation

Quick reference

Key terms – exam-ready definitions

Term	Definition (mark scheme style)
Character set	A collection of characters that a computer can recognise, where each character is assigned a unique binary code
ASCII	American Standard Code for Information Interchange; a 7-bit character set with 128 characters, mainly for English text

Term	Definition (mark scheme style)
Extended ASCII	An 8-bit version of ASCII with 256 characters, including accented letters and additional symbols
Unicode	A universal character set using 16+ bits that can represent characters from all world languages, plus emojis and symbols
UTF-16	A common Unicode encoding that uses 16 bits (2 bytes) per character
Character code	The unique binary number assigned to a character in a character set
Backwards compatible	A newer system that can work with data from an older system (Unicode includes ASCII)

ASCII vs Unicode — comparison table

Aspect	ASCII	Unicode
Bits	7 (standard) or 8 (extended)	16 or more
Characters	128 or 256	140,000+
Languages	English (+ some European)	All world languages
Emojis	No	Yes
File size	Smaller	Larger (2× or more)
Use case	English-only documents, code	International content, web

Powers of 2 — essential values

2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	2^{16}
16	32	64	128	256	512	1024	65,536

Typical pitfalls

Pitfall 1 — Forgetting "unique binary code" in definitions.
Wrong: "A character set is a list of characters."
Right: "A character set is a list of characters where **each character is assigned a unique binary code**."

Pitfall 2 — Confusing bits with characters.
Wrong: "ASCII has 128 bits."

Right: "ASCII uses 7 **bits** to represent 128 **characters**."

Pitfall 3 — Not showing working in calculations.
Wrong: "8 bits" (answer only)
Right: "2^7 = 128 (too small), 2^8 = 256 (big enough), so **8 bits**."

Pitfall 4 — Thinking ASCII and Unicode are incompatible.
Wrong: "ASCII files cannot be read by Unicode systems."
Right: Unicode is **backwards compatible** with ASCII. The first 128 Unicode characters are identical to ASCII.

Pitfall 5 — Saying Unicode is "better" without qualification.
Wrong: "Unicode is better than ASCII."
Right: "Unicode supports more characters but creates **larger files**. ASCII is suitable for English-only content where smaller file size is preferred."

Pitfall 6 — Forgetting that uppercase and lowercase have different codes.
Wrong: "A and a have the same ASCII code."
Right: 'A' = 65, 'a' = 97. They have **different codes** — there is a gap of 32.

Pitfall 7 — Not converting between bits and bytes.
Wrong: "2000 characters × 8 bits = 16,000 bytes"
Right: "2000 × 8 = 16,000 **bits** = 16,000 ÷ 8 = **2000 bytes**"

Pitfall 8 — Thinking ASCII only has letters.
Wrong: "ASCII contains 26 letters."
Right: ASCII contains 128 characters including **uppercase letters, lowercase letters, digits, punctuation, and control characters**.

Pitfall 9 — Using the wrong formula for minimum bits.
Wrong: "150 characters needs 150 bits."
Right: Find the smallest power of 2 ≥ 150. 2^8 = 256 ≥ 150, so **8 bits** are needed.

Pitfall 10 — Not applying answers to the scenario.
Wrong: "Unicode has more characters." (generic)
Right: "Unicode is needed because the Japanese website requires **kanji characters and emojis** that ASCII cannot represent."

Assessment Hub Long-Answer Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.